JALSETU — 4-Member Implementation Guide

A practical build manual for implementing every major feature, dividing work between 2 frontend members and 2 backend members.

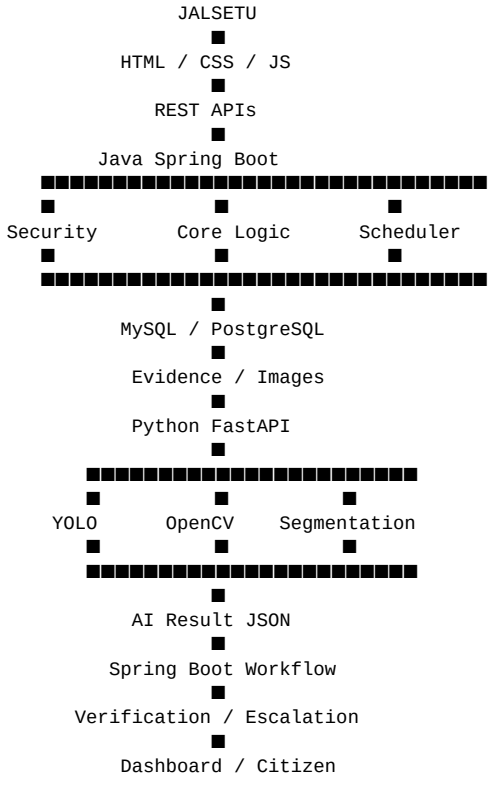
0. Project Goal and Team Structure

JALSETU is an AI-assisted pond action and accountability system. The system takes a pond problem from reporting through classification, action assignment, field work, evidence submission, AI-assisted verification, community verification, escalation and recovery tracking.

```
TEAM
■
■■■ MEMBER 1 — Frontend: Citizen / Public
■
■■■ MEMBER 2 — Frontend: Panchayat / Worker / Admin
■
■■■ MEMBER 3 — Backend: Java / Spring Boot / Database
■
■■■ MEMBER 4 — AI Backend: Python / FastAPI / Computer Vision
```

> Core principle: AI is an evidence-assistance layer. It should not be presented as an infallible authority. In particular, a single RGB image cannot scientifically prove chemical pollution, bacterial contamination, mosquito breeding, or water stagnation.

1. Final Architecture



Frontend is intentionally HTML/CSS/JavaScript rather than a frontend framework. JavaScript handles API calls, form submission, dynamic tables, status updates and dashboard charts.

2. Technology Stack and Responsibilities

Layer	Technology	Owner	Purpose
Frontend	HTML + CSS + JavaScript	M1 + M2	Citizen, worker and Panchayat interfaces

Layer	Technology	Owner	Purpose
Backend	Java + Spring Boot	M3	Business logic and REST APIs
Security	Spring Security + JWT	M3	Authentication and role-based access
ORM	Spring Data JPA / Hibernate	M3	Database persistence
Database	PostgreSQL or MySQL	M3	Users, ponds, problems, tasks, evidence
AI service	Python + FastAPI	M4	Expose AI inference APIs
Computer vision	YOLO + OpenCV + NumPy	M4	Detection, segmentation, comparison
Charts	Chart.js	M2	Dashboard analytics
Map	Leaflet	M1/M2	Pond location/map
Version control	Git + GitHub	All	Branches, pull requests, integration

3. Shared Project Setup — ALL 4 MEMBERS

3.1 Create the repository

```
JALSETU/
■■■ frontend/
■■■ backend/
■■■ ai-service/
■■■ database/
■■■ docs/
■■■ README.md
```

Create protected main branch. Each member works in a feature branch and opens a pull request. Never directly overwrite another member's module.

3.2 Git branch convention

```
main
develop
feature/frontend-citizen
feature/frontend-dashboard
feature/backend-core
feature/ai-service
```

Commit small working changes. Example: feat: add citizen problem report form. Pull before starting work and pull again before opening a PR.

3.3 Local ports

```
Frontend        http://localhost:5500
Spring Boot     http://localhost:8080
AI FastAPI      http://localhost:8000
Database        PostgreSQL/MySQL local port
```

The exact frontend port may vary depending on the static server used. The important part is that every team member uses the same documented API base URL.

3.4 Environment variables

```
SPRING_DATASOURCE_URL=...
SPRING_DATASOURCE_USERNAME=...
SPRING_DATASOURCE_PASSWORD=...
JWT_SECRET=...
AI_SERVICE_URL=http://localhost:8000
UPLOAD_DIR=...
```

4. MEMBER 1 — CITIZEN / PUBLIC FRONTEND

Member 1 owns the citizen-facing experience and public pond information. The member should not implement backend logic; all persistent operations go through Spring Boot APIs.

4.1 Folder structure

```
frontend/
■■■ citizen/
```

```
■   ■■■ index.html
■   ■■■ report.html
■   ■■■ reports.html
■   ■■■ pond.html
■   ■■■ verify.html
■   ■■■ login.html
■■■ css/
■   ■■■ global.css
■   ■■■ citizen.css
■   ■■■ components.css
■■■ js/
    ■■■ api.js
    ■■■ auth.js
    ■■■ report.js
    ■■■ pond.js
    ■■■ verify.js
```

4.2 Landing page

Build the public entry page with: JALSETU introduction, problem statement, workflow, major features, public pond information, login/register and report-problem call-to-action.

Keep the page informational. Do not put sensitive worker/admin information on the public page.

4.3 Login/Register UI

Create forms for email/username and password. On login, send credentials to Spring Boot. Store the returned JWT securely enough for the prototype and attach it to authenticated API requests.

```
POST /api/auth/login
Request:
{
  "email": "...",
  "password": "..."
}

Response:
{
  "token": "...",
  "role": "COMMUNITY_MEMBER",
  "userId": 12
}
```

4.4 Citizen dashboard

Show the citizen's own reports, current status, pond name, problem category, date and latest workflow stage.

```
REPORT STATUS EXAMPLE

Submitted
   ↓
AI Classification
   ↓
Task Assigned
   ↓
In Progress
   ↓
Verification
   ↓
Verified / Reopened
```

4.5 Problem reporting

Create a form with pond selection, category, description, image upload and location. Validate file type and size before sending.

```
POST /api/problems
multipart/form-data:
pondId
description
category
latitude
longitude
image
```

The frontend should show upload progress and a success screen containing the newly created problem ID.

4.6 Public pond profile

Display pond photo, name, location, active problems, recovery percentage and public timeline. Do not expose private worker information.

4.7 Community verification

When a task reaches COMMUNITY_VERIFICATION, display the before/after evidence and ask the community member whether the visible issue appears resolved.

```
POST /api/community-verifications

{
  "taskId": 41,
  "decision": "CONFIRM",
  "comment": "Waste appears removed"
}
```

For dispute: allow a reason and optional image. The backend should create a REOPENED state rather than deleting the original evidence.

4.8 Frontend validation and error handling

- Do not submit empty required fields.
- Reject unsupported image formats.
- Show API loading states.
- Show useful messages for 401/403/404/500 responses.
- Prevent duplicate form submissions.
- Never trust frontend role checks alone; backend authorization is mandatory.

4.9 Member 1 completion checklist

- Landing page works.
- Login works with backend.
- Citizen dashboard loads real API data.
- Problem report reaches database.
- Image upload works.
- Citizen can view status.
- Community verification works.
- Dispute/reopen request reaches backend.
- Mobile/responsive layout is usable.
- All API calls are documented.

5. MEMBER 2 — PANCHAYAT / WORKER / ADMIN FRONTEND

Member 2 owns operational interfaces: Panchayat dashboard, task management, worker screens, escalation view and analytics.

5.1 Folder structure

```
frontend/
■■■ admin/
■   ■■■ dashboard.html
■   ■■■ ponds.html
■   ■■■ problems.html
■   ■■■ tasks.html
■   ■■■ escalations.html
■   ■■■ analytics.html
■■■ worker/
■   ■■■ tasks.html
■   ■■■ task-details.html
■   ■■■ completion.html
■■■ js/
    ■■■ dashboard.js
    ■■■ ponds.js
```

```
    ■■■ tasks.js
    ■■■ worker.js
    ■■■ analytics.js
```

5.2 Panchayat dashboard

Fetch summary metrics from Spring Boot instead of hard-coding them.

```
GET /api/dashboard/summary

{
  "totalPonds": 12,
  "activeProblems": 18,
  "inProgress": 7,
  "overdue": 4,
  "awaitingVerification": 5
}
```

Render metric cards and lists of urgent tasks.

5.3 Pond management

Implement add, view and edit pond screens. The backend owns validation and persistence.

```
POST /api/ponds
GET /api/ponds
GET /api/ponds/{id}
PUT /api/ponds/{id}
```

5.4 Problem management

Create a table with filtering by pond, category, priority and status. Clicking a problem opens its full history and current task.

5.5 Task assignment UI

Flow: select problem → see recommended action → select team/worker → choose deadline → assign.

```
POST /api/tasks

{
  "problemId": 14,
  "assignedWorkerId": 8,
  "deadline": "2026-10-03T17:00:00",
  "priority": "HIGH"
}
```

5.6 Worker task UI

Workers should see only their assigned tasks. Each task shows problem, pond, deadline, instructions and upload-evidence action.

Completion screen should require before/after evidence where applicable. The worker submits a completion claim; the backend then moves the task into AI verification.

5.7 Escalation UI

Show overdue tasks, days overdue, current responsible person/team and escalation level. Provide a controlled Escalate action where manual escalation is allowed.

5.8 Analytics

Use Chart.js for problems by category, completed vs overdue, recovery over time, recurring problems and AI verification outcomes.

Every chart should use data fetched from an API endpoint. Do not calculate authoritative metrics only in browser JavaScript.

5.9 Map

Use Leaflet to show registered ponds from latitude/longitude. Clicking a pond opens its public/authorized profile.

5.10 Member 2 completion checklist

- Panchayat dashboard uses live API data.
- Ponds can be managed.
- Problems can be filtered.
- Tasks can be assigned.
- Workers can see assigned tasks.
- Evidence can be uploaded.
- Overdue tasks are visible.
- Escalations are visible.
- Charts use backend data.
- Map displays real pond coordinates.

6. MEMBER 3 — JAVA SPRING BOOT CORE BACKEND

Member 3 owns the source of truth for application state. The backend decides what a user is allowed to do and how a problem moves through the workflow.

6.1 Spring Boot package structure

```
backend/src/main/java/.../
■■■ auth/
■■■ user/
■■■ pond/
■■■ problem/
■■■ task/
■■■ evidence/
■■■ verification/
■■■ escalation/
■■■ notification/
■■■ recovery/
■■■ dashboard/
■■■ common/
```

6.2 Database schema

```
users
ponds
problems
tasks
assignments
evidence
verifications
community_verifications
escalations
notifications
recovery_metrics
pond_history
```

Use foreign keys and indexes for pondId, problemId, taskId, assignedWorkerId, status and createdAt. Do not duplicate the same business data across unrelated tables.

6.3 Entity implementation

Create JPA entities: User, Pond, Problem, Task, Assignment, Evidence, Verification, CommunityVerification, Escalation, Notification and RecoveryMetric.

Use enums for role, problem category, priority and workflow status. Avoid storing arbitrary status strings everywhere.

6.4 Authentication

Implement Spring Security with password hashing and JWT. Passwords must never be stored as plain text.

```
POST /api/auth/register
POST /api/auth/login
GET  /api/users/me
```

After login, the JWT identifies the user. Every protected endpoint checks authentication and role permissions.

6.5 Pond APIs

```
POST   /api/ponds
GET    /api/ponds
GET    /api/ponds/{id}
PUT    /api/ponds/{id}
DELETE /api/ponds/{id}
```

Validate coordinates, required name and authorized role. Keep delete operations restricted; in many cases soft deletion is safer than permanent deletion.

6.6 Problem APIs

```
POST /api/problems
GET  /api/problems
GET  /api/problems/{id}
PUT  /api/problems/{id}
```

When a problem is created, save the report and evidence first. Then trigger AI classification asynchronously or through a controlled service call. Do not let an AI outage destroy the original citizen report.

6.7 Task engine

A task belongs to a problem. Store responsible worker/team, deadline, priority and status. A task must have a clear audit history.

```
PENDING
  ↓
ASSIGNED
  ↓
IN_PROGRESS
  ↓
COMPLETION_CLAIMED
  ↓
AI_VERIFICATION
  ↓
COMMUNITY_VERIFICATION
  ↓
VERIFIED
```

Alternative paths:

```
COMMUNITY_VERIFICATION → REOPENED
ACTIVE TASK + DEADLINE PASSED → OVERDUE
OVERDUE → ESCALATED
```

6.8 Evidence API

Store evidence metadata in the database and actual files in a controlled file/object-storage location. Do not store large image binaries directly in ordinary relational columns for the prototype unless there is a specific reason.

```
POST /api/tasks/{id}/evidence
GET  /api/tasks/{id}/evidence
```

Evidence metadata should include type, uploadedBy, timestamp, taskId, optional location and file reference.

6.9 AI integration

Spring Boot sends an image or evidence references to the Python service. The AI service returns structured JSON. Spring Boot stores the result and moves the workflow forward.

```
POST http://localhost:8000/ai/verify-completion

{
  "taskId": 41,
  "taskType": "PLASTIC_REMOVAL",
  "beforeImage": "...",
  "afterImage": "..."
}
```

The Java backend should validate the returned task ID and expected task type before accepting the result.

6.10 Community verification

```
POST /api/community-verifications
```

```
GET  /api/tasks/{id}/community-verifications
```

Prevent a worker from verifying their own work. Prevent duplicate votes by the same community account for the same verification cycle.

6.11 Escalation scheduler

Use Spring Scheduler to periodically find tasks whose deadlines have passed.

```
every scheduled run:
    find active tasks
    if now > deadline:
        mark OVERDUE
        create notification
        if escalation rule satisfied:
            create escalation
```

Make the scheduler idempotent so it does not create hundreds of duplicate notifications on every run.

6.12 Notifications

Start with in-app notifications. Email/push/SMS can be added later.

```
Notification
■■■ id
■■■ recipientId
■■■ type
■■■ message
■■■ read
■■■ createdAt
```

6.13 Recovery Index

Calculate recovery from tracked intervention tasks, not from a claim that the entire ecosystem is healthy.

```
PRI = Σ(weight × completion) / Σ(weight)
```

Use verified completion as the strongest completion state. Document the weights and treat them as configurable prototype assumptions until calibrated with field observations.

6.14 Core backend tests

- Unauthorized user cannot create admin actions.
- Worker cannot edit another worker's task.
- Citizen cannot access private worker data.
- Overdue task changes state correctly.
- Reopened task retains original history.
- Duplicate community verification is blocked.
- AI failure does not delete a report.
- Invalid pond/task IDs return appropriate errors.

7. MEMBER 4 — PYTHON AI / COMPUTER VISION

Member 4 owns model preparation, inference APIs, visual comparison and AI-derived metrics. This member must work closely with Member 3 because the AI output becomes part of the backend workflow.

7.1 AI service structure

```
ai-service/
■■■ app/
■   ■■■ main.py
■   ■■■ routes/
■   ■■■ schemas/
■   ■■■ services/
■   ■■■ utils/
■■■ models/
■■■ detection/
■■■ segmentation/
■■■ verification/
■■■ stagnation/
■■■ recurrence/
```

7.2 Build the first AI feature first: waste detection

Do not start with the hardest stagnation model. First prove that one image can be processed end-to-end.

```
Image
 ↓
Preprocessing
 ↓
YOLO inference
 ↓
Detected objects / masks
 ↓
JSON result
 ↓
FastAPI
 ↓
Spring Boot
```

Train/fine-tune on suitable waste imagery and then collect pond-specific images for validation. Label relevant classes consistently.

7.3 Waste detection output

```
{
  "category": "plastic_waste",
  "count": 32,
  "debris_area_percent": 14.2,
  "confidence": 0.89
}
```

For segmentation, calculate debris area from mask pixels. For detection-only models, object count can be reported, but area estimates are less direct.

7.4 Before/after verification

The verification service receives two images. Run the same model under comparable settings on both images and compare detected count/area. Where appropriate, align images before pixel-level comparison.

```
before_count = 42
after_count = 7
reduction = (before_count - after_count) / before_count * 100
→ 83.3%
```

Do not claim that the exact percentage represents all physical waste. It represents the model's estimated visible change under the captured conditions.

7.5 Image alignment

For repeated pond images, the camera may shift. A prototype can use OpenCV feature matching such as ORB and homography to align images before comparing masks.

```
Image A + Image B
      ↓
Feature matching
      ↓
Homography
      ↓
Perspective alignment
      ↓
Mask comparison
```

7.6 Persistent standing-water analysis

Stagnation is temporal, so collect several observations from approximately the same viewpoint. Segment visible water and calculate water coverage for each observation.

```
Water coverage(t)
= water pixels / valid pond pixels × 100
```

Then compare water extent over time. Useful indicators include water coverage variation, mask overlap/IoU, boundary movement, persistent visible debris and persistent vegetation/algae-like regions.

```
IoU = intersection(maskA, maskB)
      / union(maskA, maskB)
```

> Stable water coverage is not itself proof of stagnation. The output should be named a persistent-standing-water or stagnation-risk indicator and should recommend field verification.

7.7 Example risk model

For the prototype, combine normalized indicators using configurable weights.

```
SRP = 0.40S + 0.25B + 0.20D + 0.15V

S = water stability
B = low boundary movement
D = persistent debris
V = persistent vegetation/surface indicator
```

Example result: SRP = 84%. Explain this as an 84% risk indicator, not as 84% of the pond being stagnant. The weights must eventually be calibrated against field observations.

7.8 AI FastAPI endpoints

```
POST /ai/classify
POST /ai/detect-waste
POST /ai/verify-completion
POST /ai/analyze-standing-water
GET  /health
```

Define Pydantic request/response schemas. Return predictable JSON and HTTP error codes. Never return an unstructured block of text that the Java backend must parse.

7.9 AI model evaluation

Use a held-out test set. For detection report precision, recall and mAP. For segmentation report IoU and Dice. For the standing-water indicator, compare model output with field observations and report agreement/error rather than inventing scientific accuracy.

Test across lighting, rain, reflections, camera angle, water colour, vegetation and different pond types.

7.10 AI limitations to document

- False positives and false negatives are possible.
- Image quality and lighting affect detection.
- Reflections can resemble objects or vegetation.
- A single image cannot establish temporal stagnation.
- RGB images cannot establish chemical or bacterial water quality.
- Mosquito breeding should not be claimed from ordinary pond images.
- Community and worker evidence can still be manipulated, so audit/manual review remain important.

8. INTEGRATION CONTRACT BETWEEN MEMBER 3 AND MEMBER 4

The two backend members must agree on API contracts before implementing the integration.

```
Spring Boot → FastAPI

POST /ai/classify
POST /ai/verify-completion
POST /ai/analyze-standing-water

FastAPI → Spring Boot

{
  "success": true,
  "modelVersion": "v1",
  "taskId": 41,
  "result": {...},
  "confidence": 0.89,
  "warnings": []
}
```

Always include modelVersion and warnings so future model changes can be traced.

9. COMPLETE END-TO-END IMPLEMENTATION FLOW

```
1. Citizen logs in
```

```
        ↓
2. Citizen selects pond
        ↓
3. Citizen submits problem + image
        ↓
4. Spring Boot stores report
        ↓
5. Spring Boot sends image to AI
        ↓
6. AI returns classification
        ↓
7. Backend creates/recommends action
        ↓
8. Panchayat assigns worker/team
        ↓
9. Deadline is stored
        ↓
10. Worker sees task
        ↓
11. Worker performs field work
        ↓
12. Worker uploads before/after evidence
        ↓
13. Backend stores evidence
        ↓
14. AI verifies visible change
        ↓
15. Task enters community verification
        ↓
16. Community confirms OR disputes
        ↓
17A. Confirm → VERIFIED
17B. Dispute → REOPENED
        ↓
18. Scheduler detects overdue tasks
        ↓
19. Escalation is created when required
        ↓
20. Recovery Index is recalculated
        ↓
21. Historical data can reveal recurrence
        ↓
22. Prevention action can be proposed
```

10. Feature-to-Owner Matrix

Feature	M1	M2	M3	M4
Login/Register UI	✓		API	
Citizen reporting	✓		API	AI classification
Pond management UI		✓	API	
Task assignment UI		✓	Core logic	
Worker task UI		✓	API	
Evidence upload UI	✓	✓	Storage/API	
Waste detection			AI API integration	✓
Before/after verification			Workflow	✓
Standing-water analysis			API integration	✓
Community verification	✓		Workflow	
Escalation dashboard		✓	Engine	
Notifications		✓	Engine	
Recovery Index		✓	Calculation	Data/metrics
Recurrence detection		✓	Historical API	✓
Maps	✓	✓	Pond API	

11. Development Order — Do Not Build Everything at Once

Phase 1 — Foundation

- M3: database + Spring Boot project + authentication + basic APIs.
- M1: landing + login + citizen dashboard shell.
- M2: Panchayat dashboard shell + worker UI shell.
- M4: Python/FastAPI setup + first YOLO inference script.

Phase 2 — Core CRUD

- M3: pond/problem/task CRUD and relationships.
- M1: report form and citizen report list.
- M2: pond/problem/task management screens.
- M4: classify/detect endpoint.

Phase 3 — Workflow

- M3: state machine, assignment, evidence metadata, scheduler.
- M2: assignment and worker completion UI.
- M1: community verification UI.
- M4: before/after verification endpoint.

Phase 4 — Advanced AI and analytics

- M4: standing-water temporal analysis.
- M4: recurrence metrics.
- M3: Recovery Index and escalation rules.
- M2: charts, maps and escalation screens.
- M1: public pond profile and timeline.

Phase 5 — Integration and testing

- Run one complete real workflow from citizen report to verified completion.
- Test rejected/reopened workflow.
- Test overdue/escalation workflow.
- Test AI service failure.
- Test role permissions.
- Test image upload limits.
- Test database recovery and duplicate requests.
- Collect sample pond images and evaluate AI results.

12. API Contract — Minimum Set

```
AUTH
POST /api/auth/register
POST /api/auth/login
GET  /api/users/me

PONDS
POST /api/ponds
GET  /api/ponds
GET  /api/ponds/{id}
PUT  /api/ponds/{id}

PROBLEMS
POST /api/problems
GET  /api/problems
GET  /api/problems/{id}

TASKS
POST /api/tasks
GET  /api/tasks
GET  /api/tasks/{id}
```

```
PUT  /api/tasks/{id}
POST /api/tasks/{id}/complete

EVIDENCE
POST /api/tasks/{id}/evidence
GET  /api/tasks/{id}/evidence

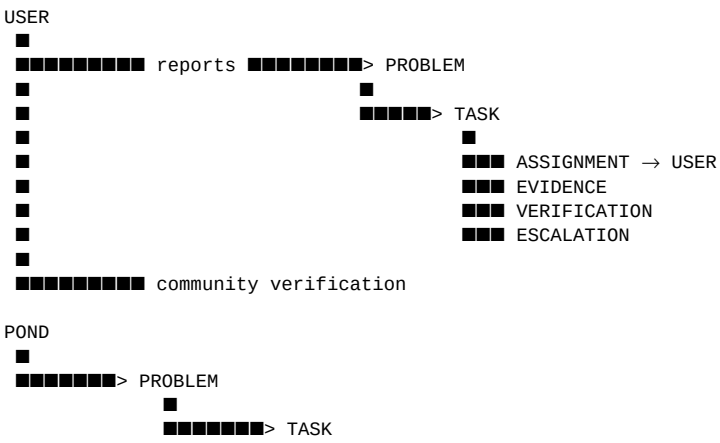
VERIFICATION
POST /api/verifications
POST /api/community-verifications

ESCALATION
GET  /api/escalations
POST /api/escalations

RECOVERY
GET /api/ponds/{id}/recovery

DASHBOARD
GET /api/dashboard/summary
```

13. Database Relationship Overview



Keep the audit trail. Never overwrite a completed task into a new unrelated state without retaining the previous event/history.

14. Testing Strategy

14.1 Frontend

- Form validation
- Responsive layouts
- API loading/error states
- Role-based page visibility
- Image preview and size validation
- Duplicate submission prevention

14.2 Backend

- Unit tests for services
- Controller/API tests
- Authorization tests
- Database integration tests
- Scheduler tests
- State-transition tests

14.3 AI

- Test inference on unseen images

- Measure precision/recall/mAP
- Measure segmentation IoU/Dice where applicable
- Test different lighting and camera angles
- Test false positives
- Test before/after edge cases
- Validate standing-water indicator against field observations

14.4 End-to-end

```
Citizen → Report
        ↓
Backend → AI
        ↓
Task → Worker
        ↓
Evidence → AI Verification
        ↓
Community
        ↓
Verified / Reopened
        ↓
Recovery / Escalation
```

15. MVP — What Must Actually Work

For a college/UHV prototype, do not try to perfect every advanced feature before the basic workflow works.

- Login with roles.
- Register/view pond.
- Citizen submits problem + image.
- AI detects at least one target category such as visible waste.
- Panchayat assigns a task.
- Worker uploads before/after evidence.
- AI produces a structured verification result.
- Community confirms or disputes.
- Overdue task becomes escalated.
- Dashboard shows current status and recovery progress.

16. Advanced Features — Add After MVP

- Persistent standing-water temporal analysis.
- Better segmentation models.
- GIS expansion.
- Mobile/offline field application.
- Push/email/SMS notifications.
- IoT water-level/turbidity/pH sensors.
- More sophisticated recurrence analysis.
- Model retraining from verified field data.
- Multi-Panchayat deployment.

17. What Each Member Should Deliver

Member 1 — Final deliverables

- Complete citizen/public UI.
- Login integration.
- Problem reporting.

- Citizen report tracking.
- Pond public profile.
- Community verification.
- Responsive styling.
- API integration documentation.

Member 2 — Final deliverables

- Panchayat dashboard.
- Pond/problem/task management screens.
- Worker task screens.
- Evidence upload UI.
- Escalation dashboard.
- Analytics/charts.
- Map view.
- API integration documentation.

Member 3 — Final deliverables

- Spring Boot project.
- Database schema.
- Authentication and JWT.
- RBAC.
- Pond/problem/task APIs.
- Workflow state machine.
- Evidence metadata handling.
- Scheduler/escalation.
- Notifications.
- Recovery calculation.
- Backend tests.
- API documentation.

Member 4 — Final deliverables

- Python/FastAPI service.
- Waste detection model.
- AI classification endpoint.
- Before/after verification.
- Image alignment/comparison.
- Standing-water temporal prototype.
- AI metrics/evaluation.
- Recurrence analysis.
- Model/API documentation.

18. Final Rule for the Team

> Do not let the project become four disconnected modules. The success condition is one complete working chain: REPORT → AI CLASSIFICATION → ACTION → ASSIGNMENT → FIELD WORK → EVIDENCE → AI VERIFICATION → COMMUNITY VERIFICATION → VERIFIED/REOPENED → ESCALATION → RECOVERY.

If that chain works reliably, the team has a functional JALSETU prototype. Advanced AI, IoT and analytics should then be layered on top.